Milestone Report: DistServe Reimplementation

Separating Prefill and Decode for LLM Serving

Systems for Deep Learning — Course Project

March 28, 2026

---

1 Project Goal

The goal of this project is to reimplement the central systems insight of **DistServe** [1]: separating LLM inference into a dedicated *prefill pool* and a dedicated *decode pool* eliminates prefill–decode interference and improves SLO goodput and tail latency under mixed workloads. We rebuild this in pure Python + PyTorch + HuggingFace (no Ray, no SwiftTransformer), targeting a 1-node / 2-GPU environment, and evaluate on real GPU hardware.

2 What Has Been Completed

2.1 System Architecture

We implement two serving designs, each runnable in two modes:

Design	Simulator	Real GPU
Colocated (baseline)	`src/simulator/baseline.py`	`src/runtime/baseline_gpu.py`
Disaggregated (DistServe-style)	`src/simulator/disaggregated.py`	`src/runtime/disaggregated_gpu.py`

Supporting modules: `src/core/{request,metrics}.py`, `src/simulator/{timing,workload,config}.py`, `src/runtime/inference_core.py`, `src/stage_b/{profile_sharegpt,fitted_timing,workload_sharegpt}.py`.

2.2 Real GPU Runtime (Primary Contribution)

`src/runtime/inference_core.py` provides wall-clock timed wrappers around HuggingFace `forward` calls with CUDA synchronization:

- `timed_prefill` — single-request prefill forward; records wall time bracketed by `torch.cuda.synchronize`.
- `timed_decode_steps` — greedy autoregressive decode loop; records per-step wall time.
- `time_transfer` — measures the wall time of moving `past_key_values` from the prefill GPU to the decode GPU via PyTorch `.to(device)` (PCIe/NVLink device-to-device copy).
- `tokenize_batch`, `timed_prefill_batch`, `timed_decode_steps_batch` — new batched variants using left-padded tokenization; group B requests into a single forward call for both prefill and each decode step.

Colocated GPU runtime (`baseline_gpu.py`): runs prefill then greedy decode on a single GPU in FIFO order, with configurable `batch_size`.

Disaggregated GPU runtime (`disaggregated_gpu.py`): prefill on GPU A, measured KV transfer

via `time_transfer`, decode on GPU B. Batching moves the full batched KV cache as a single transfer call. Both GPUs load TinyLlama/TinyLlama-1.1B-Chat-v1.0 (identical weights, separate device instances), matching the DistServe architecture of role-specific replicas.

KV cache transfer implementation: after prefill we read each transformer layer's cached keys and values from HuggingFace's `DynamicCache`, move every tensor with `.to(decode_device)`, and rebuild a fresh `DynamicCache` on the decode GPU. This is a same-node operation (both GPUs in the same process). Cross-node handoff is out of scope for this milestone and will be stated as a limitation in the final report.

2.3 Stage B: ShareGPT Profiling Pipeline

`src/stage_b/profile_sharegpt.py` measures actual GPU prefill and per-step decode latency on ShareGPT prompts, fits a linear prefill model ($a + b \cdot \text{tokens}$) via least squares, and writes `results/fitted_timing.json`. This pipeline is implemented and can be used to calibrate the simulator with real-hardware timing coefficients.

2.4 Simulator (Supporting Infrastructure)

A discrete-event simulator (`src/simulator/`) models both designs using a parametric `TimingModel`. It is used for large-scale sensitivity analysis (KV transfer cost sweep, capacity split sweep, arrival rate sweep) where real GPU runs are too slow to sweep exhaustively. Simulator results are not presented as primary evidence; they serve as a design and exploration tool.

2.5 Experiment Infrastructure

- `run_comparison.py` — core colocated vs disaggregated simulator run
- `run_ablations.py` — KV transfer cost + capacity split sweeps
- `run_load_sweep.py` — simulator goodput vs arrival rate curve
- `run_workload_mix.py` — simulator goodput vs interactive fraction
- `run_batch_sweep_gpu.py` — **real GPU** batch-size sweep (batch_size $\in \{1, 2, 4, 8\}$ for both designs)
- `run_milestone.sh` — master script; auto-detects GPU count, runs all experiments, writes results to `results/milestone/`

All outputs are timestamped CSVs + JSON metadata for reproducibility.

3 Real GPU Evidence

Setup: TinyLlama/TinyLlama-1.1B-Chat-v1.0, 2-GPU node, 30 ShareGPT-backed requests, `batch_size` = 4. TTFT SLO = 2.0 s, TPOT SLO = 0.05 s. Result file: `results/20260328T000851Z-gpu-comparis`
Per-request data: `results/20260328T000851Z-gpu-{colocated,disaggregated}-requests.csv`
(screenshot attached as evidence).

Metric	Colocated	Disaggregated	Change
Mean TTFT (s)	3.994	3.414	**−14.5%**
p50 TTFT (s)	3.506	2.925	−16.7%
p95 TTFT (s)	7.256	6.675	−8.0%
Mean E2E latency (s)	4.347	3.766	−13.4%
p99 E2E latency (s)	8.112	7.516	−7.3%
Throughput (req/s)	3.686	3.977	**+7.9%**
SLO Goodput	0.40	0.40	tied

Interpretation. The disaggregated design reduces mean TTFT by 14.5% and improves throughput by 7.9% over the colocated baseline on identical requests and hardware. Both designs achieve the same SLO goodput (0.40) at this batch size and SLO threshold because the queuing effect at `batch_size` = 4 pushes mean TTFT well above the 2.0 s threshold for a large fraction of requests in both cases. The raw latency reduction is real and directionally consistent with the DistServe claim: separating prefill from decode reduces the time until the first token is produced. An earlier sequential run (`batch_size` = 1, 15 requests) showed goodput 0.333 → 0.467 (+40%) and mean TTFT 2.629 → 2.342 s (–10.9%), confirming the direction holds across batch sizes.

TPOT. Mean TPOT is similar across designs (0.0192 s vs 0.0191 s), which is expected: once KV is on the decode GPU, per-step decode time is governed by the model and memory bandwidth, not by the prefill–decode split.

4 What Remains Before the Final Report

1. **Real GPU batch-size sweep (`run_batch_sweep_gpu.py`).** The script is implemented but not yet run. Will produce measured throughput and TTFT for `batch_size` $\in \{1, 2, 4, 8\}$ for both designs on real hardware, showing how batching interacts with the prefill–decode split.
2. **Real GPU arrival rate sweep.** The load sweep currently exists only in the simulator. Running it on real hardware — varying arrival rate across both designs and recording goodput and tail latency — is the most important remaining experiment. It would directly validate the simulator's load-collapse story with measured numbers.
3. **SLO threshold sweep.** Fix the workload and vary the TTFT SLO (e.g. 1.0 s, 1.5 s, 2.0 s, 3.0 s) to produce a goodput-vs-SLO curve for both designs. This is the canonical evaluation plot from the DistServe paper and shows the advantage holds across different strictness levels.
4. **Direct interference measurement.** Run prefill-only vs prefill+decode colocated on the same GPU and directly measure the slowdown. This empirically quantifies the interference the simulator approximates with a fixed 1.35× multiplier and removes the need for that assumption.
5. **Larger GPU runs and longer prompts.** Current GPU comparison uses 30 requests truncated to 512 tokens. Scaling to 100–200 requests with full ShareGPT prompts (1024–2048 tokens) will reduce variance, produce stable tail-latency percentiles, and stress the prefill phase more — making the KV transfer cost and interference effect more pronounced.
6. **Larger model.** TinyLlama (1.1B) has short prefill times. Running on a larger model (e.g. Llama-2-7B if GPU memory permits) would produce longer prefills, stronger interference, and a larger disaggregation benefit — closer to the real-world regime the paper targets.
7. **KV transfer microbenchmark.** Sweep prompt length vs measured `time_transfer` output to fit a `transfer_per_prompt_token` coefficient, replacing the current synthetic placeholder and closing the loop between the simulator and the GPU runtime.
8. **Stage B timing calibration.** Run `profile_sharegpt.py` to produce `fitted_timing.json` and rerun simulator sweeps with measured timing coefficients, grounding the simulator curves in real GPU numbers.
9. **Plots and figures.** Convert result CSVs into matplotlib figures: goodput vs arrival rate, goodput vs SLO threshold, TTFT CDF, batch-size throughput bar chart. These will be the primary visual evidence in the final report.

5 On Track / Blockers

On track. The full real-GPU pipeline is working end-to-end: prefill, KV transfer, and decode are all measured on real hardware. The batching extension is implemented and tested. The experiment infrastructure is automated and ready to produce the remaining GPU runs.

No hard blockers. All remaining work is execution and analysis: run more GPU experiments, calibrate timing, generate figures. No architectural changes are needed.

Scope boundary. KV transfer is same-node only (device-to-device via `.to()`). Multi-node / NCCL-based handoff is out of scope and will be documented as a limitation.

Code structure:

- `src/runtime/` — real-GPU runtime with batching support
- `src/simulator/` — discrete-event simulator (supporting tool)
- `src/stage_b/` — ShareGPT profiling and timing fitting
- `src/experiments/` — five experiment scripts + master runner
- `results/` — timestamped per-run CSVs and JSON summaries
- `results/milestone/` — named milestone outputs
- `tests/` — correctness tests (simulator invariants, timing fit)

References

[1] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. DistServe: Disaggregating Prefill and Decoding for Goodput-Optimized Large Language Model Serving. In *OSDI*, 2024.